Quizzes & Assessments — Git & GitHub for DevOps

5 Session Quizzes + 1 Final Comprehensive Exam

How to Use These Assessments

- **Quiz time:** 10 minutes per session quiz, 30 minutes for final
- **Format:** Multiple choice + practical scenarios
- **Grading:** Immediate self-check (answers at bottom of each section)
- **Passing:** 70% on quizzes, 80% on final

SESSION 1 QUIZ: Your First Repository

Time: 10 minutes | **Questions:** 8

Question 1

What does `git init` do?

- A) Downloads code from GitHub
- B) Creates a new `.git` folder to start tracking a directory
- C) Commits all files in the folder
- D) Connects to a remote server

Question 2

What is the staging area?

- A) A folder where Git stores old commits
- B) A temporary holding area for files before they are committed
- C) The GitHub website
- D) A backup folder

Question 3

Which command puts a file into the staging area?

- A) `git commit`
- B) `git push`

- C) `git add`
- D) `git init`

Question 4

Which command permanently saves your staged files?

- A) `git add`
- B) `git save`
- C) `git commit`
- D) `git stage`

Question 5

What happens if you run `git status` in a folder that is NOT a Git repository?

- A) It shows "nothing to commit"
- B) It shows "fatal: not a git repository"
- C) It creates a repository automatically
- D) It lists all files in the folder

Question 6

Which of these is a GOOD commit message?

- A) "asdf"
- B) "fix stuff"
- C) "Add nginx configuration for production"
- D) "update"

Question 7

What does `git log --oneline` show?

- A) All deleted files
- B) A compact list of commits
- C) All branches
- D) All remote servers

Question 8 (Practical Scenario)

You create a file `config.txt` and want to commit it. What is the correct sequence?

Write the commands in order:

```
1. _____
2. _____
3. _____
```

Answers: Session 1

1. **B** — `git init` creates the `.git` folder and starts tracking the directory.
2. **B** — The staging area temporarily holds files before they are committed.
3. **C** — `git add` stages files.
4. **C** — `git commit` saves staged files permanently.
5. **B** — Git shows "fatal: not a git repository" if `.git` doesn't exist.
6. **C** — Descriptive messages explain what changed and why.
7. **B** — `git log --oneline` shows each commit as one line.
8. Solution:
   ```bash
9. git add config.txt
10. git commit -m "Add configuration file"
   ```

11. git log --oneline
    ```

---

---
    ```

SESSION 2 QUIZ: Branching and Merging

Time: 10 minutes | **Questions:** 8

Question 1

Why do we use branches?

- A) To make Git run faster
- B) To protect `main` while we experiment safely
- C) To delete old commits
- D) To connect to GitHub

Question 2

Which command creates a new branch?

- A) `git create branch-name`
- B) `git branch branch-name`
- C) `git new branch-name`
- D) `git switch branch-name`

Question 3

Which command switches to an existing branch?

- A) `git branch branch-name`
- B) `git change branch-name`

- C) `git switch branch-name`
- D) `git move branch-name`

Question 4

What is a fast-forward merge?

- A) A merge that deletes the branch being merged
- B) A merge where `main` hasn't changed since branching — Git just moves the pointer
- C) A merge that runs faster than normal
- D) A merge that doesn't require a commit message

Question 5

What happens if you delete a branch after merging?

- A) All its commits are lost
- B) Its commits are safely in `main`; the branch label is deleted
- C) The merge is undone
- D) Git shows an error

Question 6

Create and switch to a branch in one command:

- A) `git branch --create branch-name`
- B) `git switch -c branch-name`
- C) `git checkout --new branch-name`
- D) `git branch -s branch-name`

Question 7

You switch to `main` and run `ls`. A file from your feature branch is NOT there. Why?

- A) The file was deleted
- B) Branches are isolated — changes on one branch don't appear on another until merged
- C) Git is broken
- D) The file is hidden

Question 8 (Practical Scenario)

You need to add a new `contact.txt` page via a branch. Write the complete sequence:

```
1. _____
2. _____
3. _____
4. _____
5. _____
6. _____
7. _____
```

Answers: Session 2

1. **B** — Branches let you work safely without affecting `main`.
2. **B** — `git branch branch-name` creates a branch.
3. **C** — `git switch branch-name` changes branches.
4. **B** — Fast-forward happens when `main` hasn't diverged.
5. **B** — Commits stay in history; only the branch label is deleted.
6. **B** — `git switch -c branch-name` creates and switches.
7. **B** — Branches are isolated by design.

8. Solution:
   ```bash
9. git branch feature-contact
10. git switch feature-contact
11. echo "Email: hello@example.com" > contact.txt
12. git add contact.txt
13. git commit -m "Add contact page"
14. git switch main
15. git merge feature-contact
16. git branch -d feature-contact
    ```

SESSION 3 QUIZ: GitHub, Push, Pull, and Pull Requests

Time: 10 minutes | **Questions:** 8

Question 1

What is a remote?

- A) A git command that undoes changes
- B) A copy of your repository on another server (like GitHub)
- C) A branch that is far away from `main`
- D) A deleted commit

Question 2

Which command uploads your commits to GitHub?

- A) `git upload`
- B) `git send`
- C) `git push`
- D) `git publish`

Question 3

Which command downloads the latest changes from GitHub?

- A) `git download`
- B) `git fetch`

- C) `git pull`
- D) `git receive`

Question 4

What does `git remote add origin ...` do?

- A) Deletes the remote
- B) Creates a nickname `origin` for your GitHub repo URL
- C) Pushes code to GitHub
- D) Creates a new branch called `origin`

Question 5

What is a Pull Request?

- A) A request to delete a branch
- B) A formal request to merge your branch into main, with code review
- C) A command to download code
- D) An email to your manager

Question 6

Why shouldn't you push directly to `main`?

- A) It's faster to use branches
- B) Direct pushes bypass code review and can break production
- C) GitHub doesn't allow it
- D) Branches cost money

Question 7

What does `git clone` do?

- A) Deletes a repository
- B) Copies a remote repository to your local machine
- C) Creates a new empty repository
- D) Merges two branches

Question 8 (Practical Scenario)

You just merged a PR on GitHub. Your local `main` doesn't have the changes yet. Write the commands to sync:

```
1. _____
2. _____
3. _____
```

Answers: Session 3

1. **B** — A remote is a copy on a server.
2. **C** — `git push` uploads commits.
3. **C** — `git pull` downloads and merges; `git fetch` only downloads.
4. **B** — `origin` is a nickname for the remote URL.
5. **B** — PRs request merging with review.
6. **B** — Direct pushes skip quality checks.
7. **B** — `git clone` copies a remote repo locally.
8. Solution:
   ```bash
9. git switch main
10. git pull
   ```

11. git branch -d feature-branch-name
    ```

---

---
    ```

SESSION 4 QUIZ: Undoing and Conflicts

Time: 10 minutes | **Questions:** 8

Question 1

You need to undo a bad commit that was already pushed. What do you use?

- A) `git reset --hard`
- B) `git revert`
- C) `git delete`
- D) `git erase`

Question 2

What does `git reset --soft HEAD~1` do?

- A) Deletes all files
- B) Removes the last commit but keeps the changes staged
- C) Deletes the repository
- D) Pushes to GitHub

Question 3

You see this in a file:

```
<<<<<<< HEAD
my version
=======
their version
>>>>>>> their-branch
```

What is this?

- A) A bug in Git
- B) A merge conflict that needs manual resolution
- C) A virus
- D) An encrypted message

Question 4

After fixing a merge conflict, what must you do?

- A) Delete the file
- B) Run `git restart`
- C) `git add .` then `git commit`
- D) Run `git fix`

Question 5

What is `git stash` used for?

- A) Deleting branches
- B) Saving uncommitted work temporarily so you can switch tasks
- C) Pushing to GitHub
- D) Creating a new repository

Question 6

How do you restore stashed work?

- A) `git stash restore`
- B) `git stash pop`
- C) `git stash apply`
- D) Both B and C

Question 7

What is `git reflog`?

- A) A list of remote servers
- B) A log of all Git actions (even deleted commits)
- C) A list of branches
- D) A log of GitHub API calls

Question 8 (Practical Scenario)

You committed a bad change but haven't pushed yet. Write the safe command to undo the last commit while keeping your changes:

> _____

Answers: Session 4

1. **B** — `git revert` is safe for shared history.
2. **B** — `--soft` removes the commit but keeps changes staged.
3. **B** — These markers indicate a conflict. Remove them, edit, stage, commit.
4. **C** — Stage resolved files and commit to complete the merge.

5. **B** — Stash saves work-in-progress temporarily.
6. **D** — `pop` applies and removes; `apply` applies but keeps it.
7. **B** — Reflog records everything — your safety net.
8. Solution: `git reset --soft HEAD~1`

SESSION 5 QUIZ: CI/CD with GitHub Actions

Time: 10 minutes | **Questions:** 8

Question 1

What does CI stand for?

- A) Code Integration
- B) Continuous Integration
- C) Computer Intelligence
- D) Central Integration

Question 2

GitHub Actions workflows are stored in which folder?

- A) `.github/actions/`
- B) `.github/workflows/`
- C) `.actions/`
- D) `workflows/`

Question 3

What file format are GitHub Actions workflows written in?

- A) JSON
- B) XML

- C) YAML
- D) HTML

Question 4

What does `runs-on: ubuntu-latest` mean?

- A) Your laptop must run Ubuntu
- B) GitHub will use a virtual machine running Ubuntu
- C) The workflow only works on Ubuntu
- D) The branch name must be `ubuntu-latest`

Question 5

What does `actions/checkout@v4` do?

- A) Deletes the repository
- B) Downloads your code onto the runner
- C) Creates a new branch
- D) Logs into GitHub

Question 6

What is branch protection?

- A) A firewall around your code
- B) Rules that prevent direct pushes and enforce PRs + CI
- C) A password on the repository
- D) A backup service

Question 7

Where do you store secrets (like API keys) in GitHub Actions?

- A) In the workflow file
- B) In a text file in the repo
- C) In Repo Settings → Secrets and variables
- D) In the README

Question 8 (Practical Scenario)

Write a workflow that triggers on every push to `main` and prints "Deploying to production!".

```
name: _____________________________________
on:
  _____________________________________:
    branches: [ _____________________________________ ]
jobs:
  deploy:
    runs-on: _____________________________________
    steps:
      - run: echo "Deploying to production!"
```

Answers: Session 5

1. **B** — Continuous Integration
2. **B** — `.github/workflows/`
3. **C** — YAML
4. **B** — GitHub provides a VM (runner) running Ubuntu
5. **B** — It checks out your code onto the runner
6. **B** — Rules for `main`: PRs required, CI must pass
7. **C** — Secrets belong in Settings → Secrets, NEVER in code

8. Solution:
   `yaml name: Deploy on: push: branches: [ main ] jobs: deploy: runs-on: ubuntu-latest steps: - run: echo "Deploying to production!"`

FINAL COMPREHENSIVE EXAM

Time: 30 minutes | **Questions:** 15 | **Passing:** 80%

Section A: Multiple Choice (10 questions)

Question 1

Which command initializes a new Git repository?

- A) `git start`
- B) `git init`
- C) `git new`
- D) `git create`

Question 2

What is the correct sequence?

- A) `add` → `commit` → `push`
- B) `commit` → `add` → `push`
- C) `push` → `add` → `commit`
- D) `init` → `push` → `commit`

Question 3

You want to undo a commit that is already on GitHub. You should:

- A) `git reset --hard`
- B) `git revert`
- C) Delete the repository
- D) Email GitHub support

Question 4

Two branches edited the same line. What does Git do?

- A) Automatically picks the first branch
- B) Creates a merge conflict and waits for a human
- C) Deletes both changes
- D) Emails the team

Question 5

What does `git stash pop` do?

- A) Deletes all stashes
- B) Applies the latest stash and removes it
- C) Shows stash history
- D) Pushes code to GitHub

Question 6

A Pull Request is:

- A) A request to delete a branch
- B) A formal request to merge code with review
- C) A command to download code
- D) An email to the manager

Question 7

Where are GitHub Actions workflows stored?

- A) `.github/actions/`
- B) `.github/workflows/`
- C) `actions.yml`
- D) `ci.yml`

Question 8

What does `git push -u origin main` do?

- A) Deletes the main branch
- B) Pushes main and sets upstream tracking
- C) Creates a new repository
- D) Pulls code from GitHub

Question 9

Which is the safe way to undo a local commit while keeping changes?

- A) `git reset --hard HEAD~1`
- B) `git reset --soft HEAD~1`
- C) `git rm -rf .`
- D) `git undo`

Question 10

What does branch protection prevent?

- A) Creating branches
- B) Direct pushes to main without PR or CI
- C) Opening pull requests
- D) Deleting branches

Section B: Practical Scenarios (5 questions)

Question 11

Write the complete sequence to set up a new repo, add a file, and push it to GitHub.

```
_____
_____
_____
_____
_____
_____
_____
```

Question 12

You committed a file to `main` but meant to commit to a feature branch. Write the fix.

```
_____
_____
_____
```

Question 13

Write a GitHub Actions workflow that runs on every pull request to `main` and prints "Running tests...".

```
_____
_____
_____
_____
_____
_____
_____
_____
```

Question 14

You see a merge conflict in `config.txt`. Write the exact steps to resolve it.

```
1. _____
2. _____
3. _____
4. _____
```

Question 15 (Bonus)

You accidentally committed a file called `secrets.env` containing a password. It has NOT been pushed. Write the complete fix.

> 1. _____
> 2. _____
> 3. _____
> 4. _____
> 5. _____

Final Exam Answers

Section A

Q	Answer
1	B
2	A
3	B
4	B
5	B
6	B
7	B
8	B
9	B
10	B

Section B

Question 11:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin git@github.com:USER/REPO.git
git branch -M main
git push -u origin main
```

Question 12:

```
git switch -c feature-branch  # Brings the commit
git switch main
git reset --soft HEAD~1
```

Question 13:

```
name: Test
on:
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Running tests..."
```

Question 14:

```
1. Open config.txt, remove <<<<<<< ======= >>>>>>> markers
2. Edit to keep desired content
3. git add config.txt
4. git commit -m "Resolve config conflict"
```

Question 15 (Bonus):

```
1. git reset --soft HEAD~1           # Undo last commit, keep changes staged
2. git restore --staged secrets.env  # Unstage the file
3. echo "secrets.env" >> .gitignore  # Add to .gitignore
4. git add .gitignore
5. git commit -m "Add .gitignore and remove secret"
# Then change the password immediately!
```

Scoring Guide

Score	Grade	Meaning
15/15	A+	DevOps ready
13-14	A	Strong understanding
11-12	B	Good, review weak areas
9-10	C	Needs more practice
< 9	D/F	Retake Session quizzes

Good luck!